자료구조 실습

10/14

CONTENTS

오늘 실습 순서

- 스택과 큐 복습 — LIFO / FIFO, 배열로 만들 때 필요한 인덱스
- 문제 1~3: 문제를 먼저 보고 직접 작성한 뒤, 같이 풀이 확인
  1) ArrayStack push · pop · peek
  2) evalPostfix — 스택으로 후위수식 계산기 만들기
  3) CircularQueue — 원형 큐(링버퍼) enqueue · dequeue · isFull · size
- 문제 4: 당겨서 빼는 큐 vs 원형 큐 비교 실험 — dequeue 비용을 직접 측정
- 도전 문제(선택): 중위 표기식 -> 후위 표기식 변환 (스택 + 큐 함께 쓰기)
- 연산별 Big-O 총정리

REFERENCE

스택과 큐 — 넣는 곳과 빼는 곳이 다름

- 둘 다 '한쪽 끝에서만' 다루는 리스트. 어디서 꺼내느냐만 다름

스택 vs 큐

	스택 (Stack)	큐 (Queue)
순서	LIFO — 마지막에 넣은 것이 먼저 나옴	FIFO — 먼저 넣은 것이 먼저 나옴
넣기	push : 맨 위(top)에	enqueue : 맨 뒤(rear)에
빼기	pop : 맨 위(top)에서	dequeue : 맨 앞(front)에서
필요한 인덱스	top 하나	front, rear 두 개



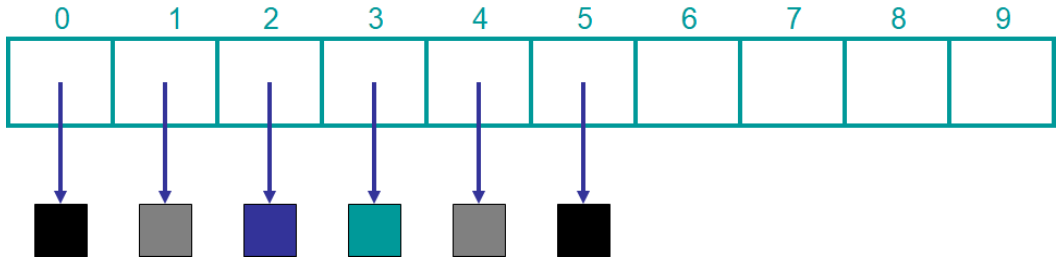




ArrayStack, CircularQueue

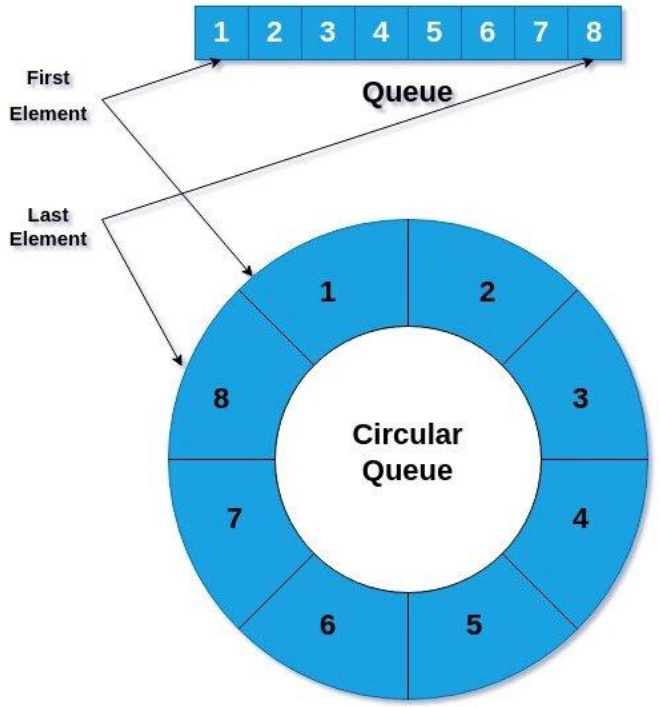
ArrayStack

```
class ArrayStack:
    def __init__(self, max_size=MAX_SIZE):
        self.data = [None] * max_size
        self.top = -1       # 맨 위 원소 인덱스
                            # -1이면 비어 있음
```



CircularQueue

```
class CircularQueue:
    def __init__(self, capacity=MAX_SIZE):
        self.cap = capacity + 1  # 한 칸 비움
        self.data = [None] * self.cap
        self.front = 0      # 맨 앞 원소 자리
        self.rear = 0       # 다음에 넣을 자리
```



push( ) · pop( ) · peek( ) — 맨 위에서만 넣고 빼기

완성할 코드 (practice.py)

```
def push(self, value):
    if self.isFull(): raise OverflowError(...)
    NEED_TO_SOLVE           # 1) top 한 칸 올리기
    NEED_TO_SOLVE           # 2) 그 자리에 value 쓰기

def pop(self):
    if self.isEmpty(): raise IndexError(...)
    value = NEED_TO_SOLVE   # 1) 맨 위 값 읽기
    self.data[self.top] = None
    NEED_TO_SOLVE           # 2) top 한 칸 내리기
    return value

def peek(self):
    ...
    return NEED_TO_SOLVE    # 읽기만, top은 그대로
```

생각해보기

```
push(7) push(3) push(6)
-> data=[7,3,6]  top=2

pop() 하면 반환값은?
그 뒤 top은?
```

⏱ 먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

top을 올리고 쓰기, 읽고 내리기

풀이 코드

```
def push(self, value):
    ...
    self.top += 1
    self.data[self.top] = value

def pop(self):
    ...
    value = self.data[self.top]
    self.data[self.top] = None
    self.top -= 1
    return value

def peek(self):
    ...
    return self.data[self.top]
```

push 7,3,6 -> pop -> push 4 예시

```
push(7)  [7]        top=0
push(3)  [7,3]      top=1
push(6)  [7,3,6]    top=2
pop()    -> 6  [7,3]   top=1
push(4)  [7,3,4]    top=2
peek()   -> 4 (top 그대로)
```

```
순서가 바뀌면? top을 올리기 전에 쓰면
맨 위 값을 덮어써 버림
```

실행 결과

```
[PASS] 문제1 push 7,3,6 -> pop -> push 4: (6, 4, [7, 3, 4])
```

REFERENCE

후위 표기식(Postfix) — 연산자를 피연산자 뒤에 쓰기

- 우리가 평소 쓰는 3 + 4 는 중위(infix). 연산자를 뒤로 보내면 3 4 + (후위)
- 후위식은 괄호와 우선순위가 필요 없음 -> 왼쪽부터 한 번만 읽으면 계산 끝

같은 식, 다른 표기

중위 (infix)	후위 (postfix)	값
3 + 4	3 4 +	7
(3 + 4) * 2	3 4 + 2 *	14
3 + 4 * 2	3 4 2 * +	11
5 + (1 + 2) * 4 - 3	5 1 2 + 4 * + 3 -	14

```
계산 규칙: 숫자는 push, 연산자를 만나면 두 개 pop 해서 계산한 뒤 결과를 push
컴파일러·계산기가 수식을 실제로 이렇게 바꿔서 계산함 (도전 문제가 그 변환 과정)
```

evalPostfix( ) — 스택으로 후위수식 계산기 만들기

완성할 코드 (practice.py)

```
def evalPostfix(expr):
    st = ArrayStack()

    for token in expr.split():
        if token in OPERATORS:
            if NEED_TO_SOLVE:   # 피연산자 2개 미만이면
                raise ValueError("피연산자가 부족함")

            b = NEED_TO_SOLVE   # 먼저 꺼낸 것 = 오른쪽
            a = NEED_TO_SOLVE   # 나중에 꺼낸 것 = 왼쪽

            NEED_TO_SOLVE       # apply 결과를 push

        else:
            NEED_TO_SOLVE       # 숫자는 int로 push

    if NEED_TO_SOLVE:           # 1개가 남지 않았으면
        raise ValueError("피연산자가 남음")

    return NEED_TO_SOLVE
```

생각해보기

```
"9 3 /" 의 답은 3
"2 7 -" 의 답은 -5
-> 꺼낸 순서와 a, b 순서 주의

"1 +"  -> 왜 오류인가?
"1 2"  -> 왜 오류인가?
```

```
apply(op, a, b)는 주어짐
나눗셈은 0 쪽으로 버림
```

⏱ 먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

문제 2 · 풀이 · evalPostfix( )

숫자는 쌓고, 연산자가 오면 두 개 꺼내 계산

풀이 코드

```
for token in expr.split():
    if token in OPERATORS:
        if st.size() < 2:
            raise ValueError(...)
        b = st.pop()
        a = st.pop()
        st.push(apply(token, a, b))
    else:
        st.push(int(token))

if st.size() != 1:
    raise ValueError(...)
return st.pop()
```

"5 1 2 + 4 * + 3 -" 추적

토큰	동작	스택 (바닥 -> 위)
5 1 2	push	5 1 2
+	1 + 2 = 3	5 3
4	push	5 3 4
*	3 * 4 = 12	5 12
+	5 + 12 = 17	17
3	push	17 3
-	17 - 3 = 14	14 -> 답

실행 결과

```
[PASS] 문제2 '5 1 2 + 4 * + 3 -': 14
[PASS] 문제2 '2 7 -' (순서 주의): -5
[PASS] 문제2 '1 +' -> ValueError: True
```

REFERENCE

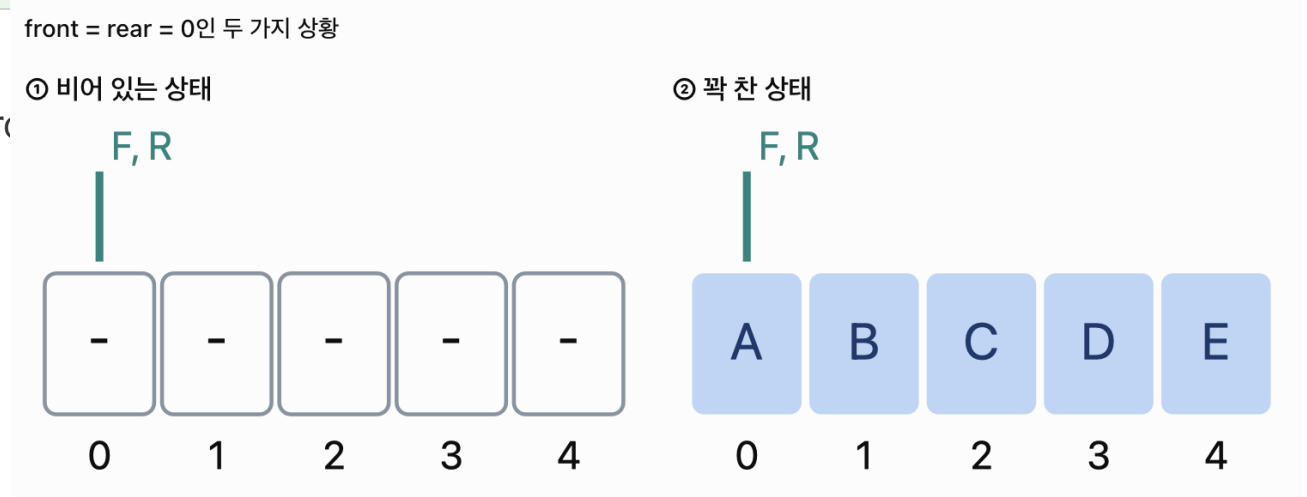
배열로 큐를 만들면 생기는 문제 — 그래서 원형 큐

세 가지 방법 비교

방법	dequeue 할 때	문제점
① 당기기 (ShiftQueue)	data[0]을 꺼내고 나머지를 한 칸씩 당김	매번 N개 이동 -> O(N)
② front만 전진	front += 1 (당기지 않음)	앞쪽 빈칸을 다시 못 씀 -> 자리가 남아도 rear가 끝에 닿으면 꽉 참
③ 원형 (링버퍼)	front = (front + 1) % cap	없음. 끝 다음은 0번 칸 -> O(1)

```
인덱스 0 1 2 3 4 를 원으로 이어 붙였다고 생각하기 (4 다음이 0)
다음 칸 = (현재 + 1) % cap   ->  배열 끝을 넘어가면 자동으로 0으로 돌아옴
```

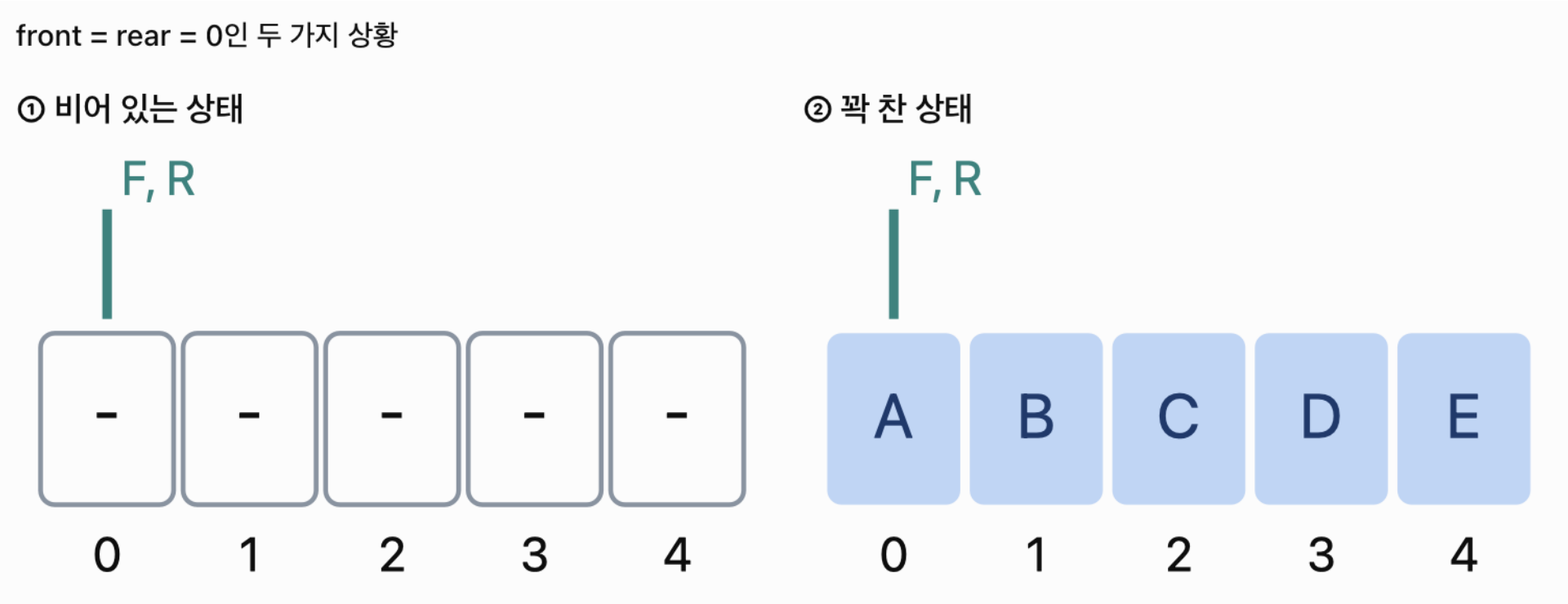
- 남은 문제: front == rear 이면 비었다는 뜻인데, 꽉 차도 fr
- 해결: 한 칸은 항상 비워 둔다 -> rear 다음 칸이 front이면



REFERENCE

배열로 큐를 만들면 생기는 문제 — 그래서 원형 큐

- 남은 문제: front == rear 이면 비었다는 뜻인데, 꽉 차도 front == rear가 됨
- 해결: 한 칸은 항상 비워 둔다 -> rear 다음 칸이 front이면 꽉 찬 것 (용량 4면 배열은 5칸)

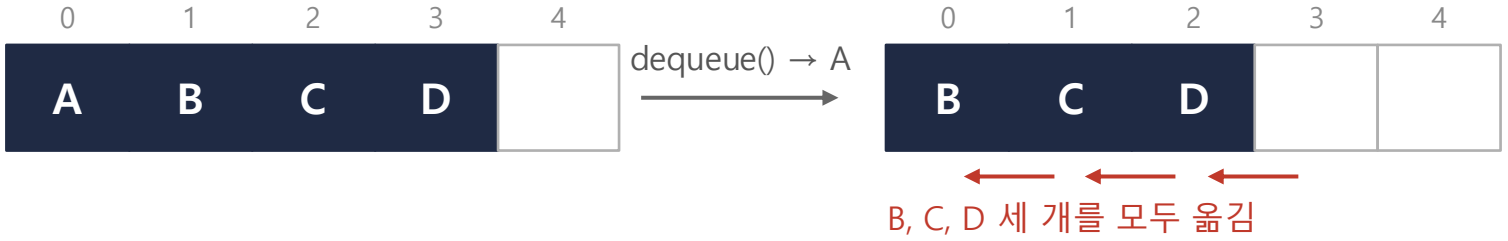


REFERENCE

당기면 O(N), 안 당기면 앞칸 낭비 — 원형으로 둘 다 해결

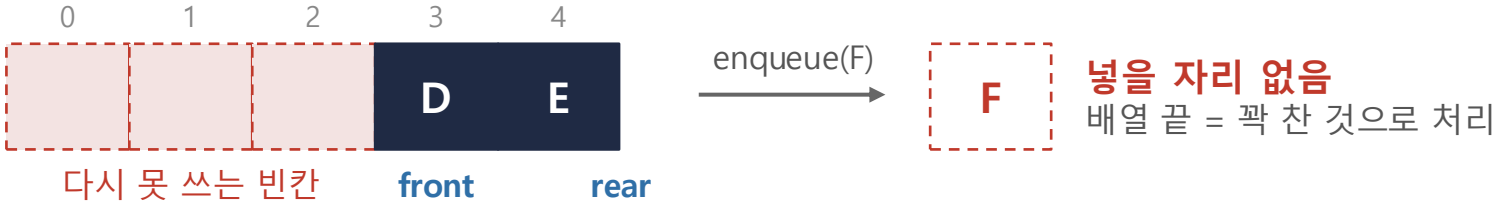
① 당기기 (ShiftQueue)

꺼낼 때마다 남은 원소를 **전부 한 칸씩 당김**
→ 원소 N개면 N번 이동, **O(N)**



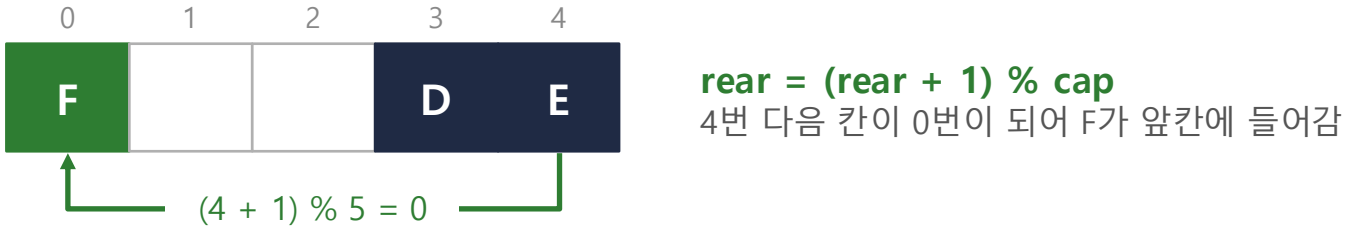
② front만 전진

앞칸이 비어도 **rear가 끝에 닿으면**
더 못 넣음 → 자리 낭비



③ 원형 (링버퍼)

끝 다음은 **0번 칸** → 앞칸 재사용
원소 이동 없이 **O(1)**



CircularQueue — 링버퍼로 enqueue / dequeue

완성할 코드 (practice.py)

```
def isFull(self):
    return (self.rear + 1) % self.cap == self.front

def size(self):
    return (self.rear - self.front + self.cap) % self.cap

def enqueue(self, value):
    if self.isFull(): raise OverflowError(...)
    self.data[NEED_TO_SOLVE] = value
    self.rear = NEED_TO_SOLVE

def dequeue(self):
    if self.isEmpty(): raise IndexError(...)
    value = self.data[NEED_TO_SOLVE]
    self.data[self.front] = None
    self.front = NEED_TO_SOLVE
    return value
```

생각해보기

```
cap = 5 (용량 4)
front=2, rear=1 일 때
원소는 몇 개인가?

힌트: % self.cap
```

⏱ 먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

isFull은 rear 다음 칸을, size는 rear - front를 본다

isFull( )

```
(self.rear + 1) % self.cap == self.front
```

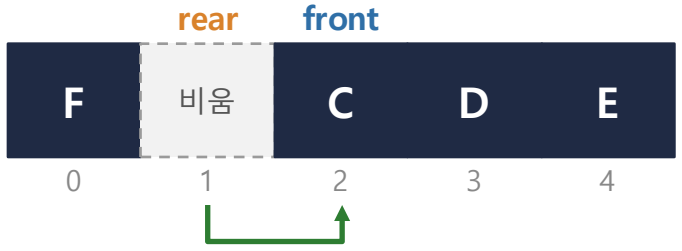
rear는 다음에 넣을 칸. **그 다음 칸이 front면 꽉 참**

예 1 · front=0, rear=4



(4 + 1) % 5 = 0
= front → **꽉 참**

예 2 · front=2, rear=1 (한 바퀴 돎)



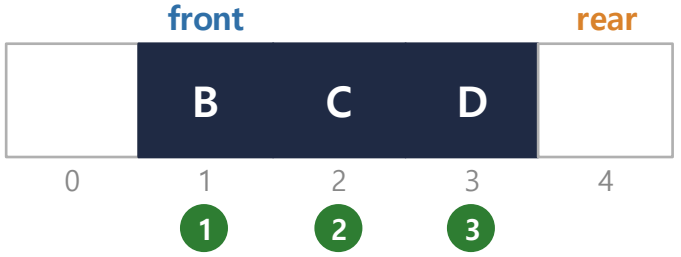
(1 + 1) % 5 = 2
= front → **꽉 참**

size( )

```
(self.rear - self.front + self.cap) % self.cap
```

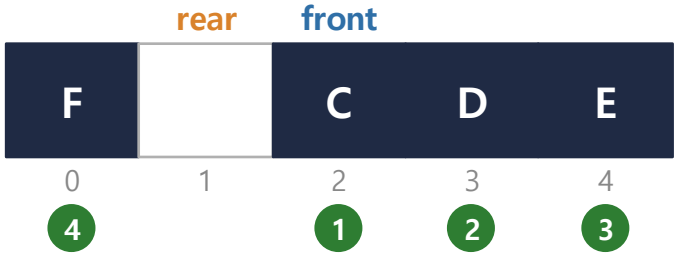
rear - front가 개수. **음수면 cap을 더해 보정**

예 1 · front=1, rear=4



(4 - 1 + 5) % 5
= 8 % 5 = **3개**

예 2 · front=2, rear=1 (한 바퀴 돎)



1 - 2 = -1 (음수)
(-1 + 5) % 5 = **4개**

CircularQueue — 링버퍼로 enqueue / dequeue

완성할 코드 (practice.py)

```
def isFull(self):
    return (self.rear + 1) % self.cap == self.front

def size(self):
    return (self.rear - self.front + self.cap) % self.cap

def enqueue(self, value):
    if self.isFull(): raise OverflowError(...)
    self.data[NEED_TO_SOLVE] = value
    self.rear = NEED_TO_SOLVE

def dequeue(self):
    if self.isEmpty(): raise IndexError(...)
    value = self.data[NEED_TO_SOLVE]
    self.data[self.front] = None
    self.front = NEED_TO_SOLVE
    return value
```

생각해보기

```
cap = 5 (용량 4)
front=2, rear=1 일 때
원소는 몇 개인가?

힌트: % self.cap
```

```
front=2, rear=1, cap=5
rear - front = -1
(-1 + 5) % 5 = 4  -> 4개

rear가 한 바퀴 돌아 front보다
작아져도 개수가 맞게 나옴
```

⏱ 먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

링버퍼 들여다보기 — dump( ) 출력

python3 practice.py 실행 시 체크포인트 다음에 출력됨

```
=== 링버퍼 들여다보기 (용량 4 -> 배열 칸은 5개, 한 칸은 항상 비움) ===
  enqueue(1)     [1 . . . .]  front=0 rear=1
  enqueue(2)     [1 2 . . .]  front=0 rear=2
  enqueue(3)     [1 2 3 . .]  front=0 rear=3
  enqueue(4)     [1 2 3 4 .]  front=0 rear=4     <- isFull() = True
  dequeue() -> 1 [. 2 3 4 .]  front=1 rear=4
  dequeue() -> 2 [. . 3 4 .]  front=2 rear=4
  enqueue(5)     [. . 3 4 5]  front=2 rear=0     <- rear가 끝을 넘어 0으로
  enqueue(6)     [6 . 3 4 5]  front=2 rear=1     <- 앞쪽 빈칸을 다시 사용
```

- 원소를 한 번도 옮기지 않았음. front, rear 두 인덱스만 움직임
- 마지막 줄: 큐 내용은 3 4 5 6 (front부터 읽음), 배열에서는 6이 맨 앞 칸에 있음
- enqueue(4) 직후 빈칸이 하나 남아 있어도 꽉 찬 것으로 봄 -> 빈 큐(front == rear)와 구분

모든 인덱스 이동에 % cap — 끝을 넘으면 0으로

풀이 코드

```
def isFull(self):
    return (self.rear + 1) % self.cap == self.front

def size(self):
    return (self.rear - self.front + self.cap) % self.cap

def enqueue(self, value):
    ...
    self.data[self.rear] = value
    self.rear = (self.rear + 1) % self.cap

def dequeue(self):
    ...
    value = self.data[self.front]
    self.data[self.front] = None
    self.front = (self.front + 1) % self.cap
    return value
```

size( ) 에서 + cap 을 하는 이유

```
front=2, rear=1, cap=5
rear - front = -1
(-1 + 5) % 5 = 4  -> 4개

rear가 한 바퀴 돌아 front보다
작아져도 개수가 맞게 나옴
```

실행 결과

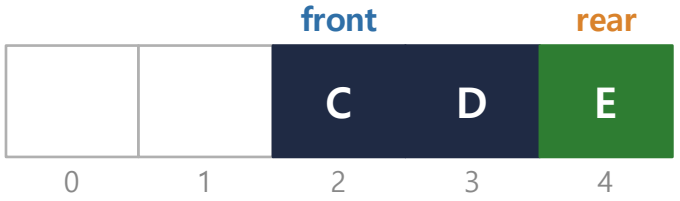
```
[PASS] 문제3 넣고 빼며 끝을
  넘어가기: (True, 1, 2,
  [3, 4, 5, 6], 4, True)
```

enqueue는 rear 칸에, dequeue는 front 칸에서 — 처리 후 한 칸 전진

enqueue(E) front=2, rear=4 에서 시작

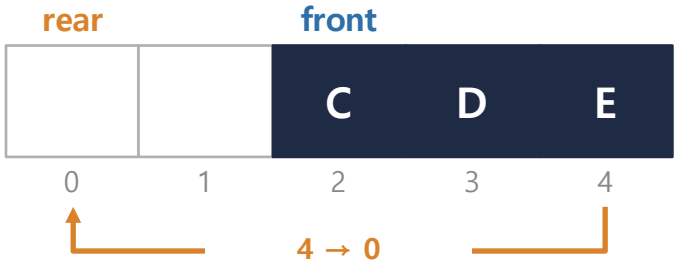
rear = 다음에 넣을 빈칸. **먼저 rear 칸에 쓰고 → rear 전진**

```
① self.data[self.rear] = value
```



data[4] = E
rear 칸에 씀

```
② self.rear = (self.rear + 1) % self.cap
```

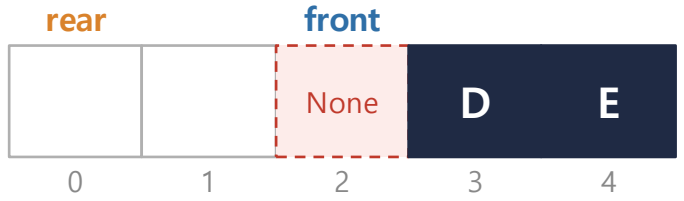


(4 + 1) % 5 = 0
rear는 0번 칸으로

dequeue( ) front=2, rear=0 에서 시작

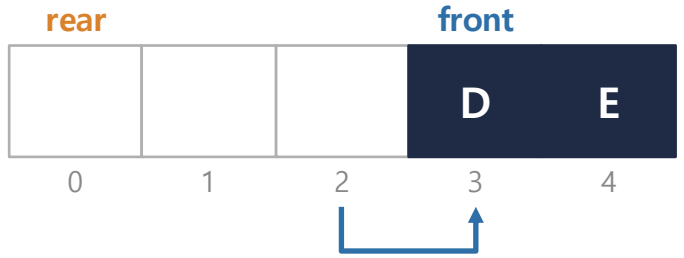
front = 맨 앞 원소 칸. **front 칸을 꺼내고 → front 전진**

```
① value = self.data[self.front]
  self.data[self.front] = None
```



C를 꺼내 반환
그 칸은 None으로 비움

```
② self.front = (self.front + 1) % self.cap
```



(2 + 1) % 5 = 3
front는 3번 칸으로

공통: 값을 쓰거나 꺼낸 다음에 인덱스를 옮긴다. 옮길 때는 항상 **(i + 1) % cap** → 4 다음은 0

실험 — dequeue + enqueue 1회: 당기기 vs 링버퍼

측정 결과 (원소 N개가 든 큐에서 1회 평균, 실행 환경마다 값은 다름)

```
       N |  ShiftQueue | CircularQueue
-----------------------------------------
    1000 |      99.2us |       0.403us
    2000 |     157.4us |       0.356us
    4000 |     312.3us |       0.360us
    8000 |     625.9us |       0.362us
   16000 |    1252.9us |       0.363us
```

```
ShiftQueue    : N이 2000 -> 16000 (8배)이면 157.4us -> 1252.9us (약 8배) -> O(N)
CircularQueue : N이 커져도 0.36~0.40us 그대로 -> O(1)
```

- 측정값은 컴퓨터마다 다름. 절대 시간이 아니라 'N이 커질 때 몇 배가 되는가'를 보는 것
- 16000개일 때 한 번 꺼내는 데 약 3,400배 차이 -> 대기열이 긴 시스템일수록 차이가 커짐

문제 4 · 해석

왜 링버퍼는 N과 상관없이 빠른가

- ShiftQueue는 앞자리를 비우기 위해 남은 원소를 전부 옮김 -> 원소 수만큼 일함
- 링버퍼는 원소는 그대로 두고 front 인덱스만 한 칸 옮김 -> 늘 같은 양의 일
- 대신 크기가 고정됨. 꽉 차면 더 못 넣음 (OverflowError)

연산별 비교

연산	ShiftQueue	CircularQueue	이기는 쪽
enqueue	O(1)	O(1)	같음
dequeue	O(N)	O(1)	CircularQueue
저장 공간	N칸	N + 1칸 (한 칸 비움)	ShiftQueue (한 칸 차이)

실제 쓰임: 키보드/네트워크 입력 버퍼, 오디오 스트리밍 버퍼, OS의 스케줄링 대기열
'최근 N개만 보관' 같은 고정 크기 기록에도 링버퍼를 씀

SUMMARY

오늘 구현한 연산의 Big-O

연산	하는 일	Big-O
`push / pop / peek`	top 위치에서 쓰기, 읽기, top 이동	O(1)
`evalPostfix`	토큰 N개를 한 번씩 읽으며 push / pop	O(N)
`enqueue / dequeue (링버퍼)`	rear / front 위치에서 쓰기, 읽기, % cap 이동	O(1)
`isFull / size (링버퍼)`	front, rear 값으로 계산만	O(1)
`dequeue (ShiftQueue)`	남은 원소 전부 한 칸씩 당기기	O(N)

```
스택과 큐는 '어디서 넣고 어디서 빼는가'를 제한한 리스트 -> 그 대신 모든 연산이 O(1)
배열로 큐를 만들 때는 원소를 옮기지 말고 인덱스를 돌린다 (% cap)
```

7주차 실습 끝

수고하셨습니다